

An Empirical Analysis of Machine Learning Algorithms for Network Intrusion Detection and a Corresponding Interactive Dashboard

GOTAM SAI VARSHITH
22BCE1605

gotamsai.varshith2022@vitstudent.ac.in

PAVAN KRISHNA R
24BCE5294

pavankrishna.r2024@vitstudent.ac.in

MRINAL SWAIN
24BCE5357

mrinal.swain2024@vitstudent.ac.in

ABSTRACT:

We are currently living in an era of unprecedented digital transformation, defined by the exponential proliferation of network-interconnected devices, cloud-based services, and the Internet of Things (IoT). This hyper-connectivity has woven networks into the very fabric of modern society, making them the indispensable backbone for critical global infrastructures, including finance, healthcare, energy, and government. The benefits of this digital ecosystem are immeasurable, but they are inextricably linked to a parallel escalation in security vulnerabilities and a vast, complex cyber landscape. Concurrently, the actors perpetrating cyber threats have evolved from isolated individuals to sophisticated, well-funded organizations and state-sponsored groups. These adversaries employ highly advanced, automated, and persistent attack methodologies. The contemporary threat landscape is dominated by Advanced Persistent Threats (APTs), polymorphic and metamorphic malware that changes its own code to evade detection, and, most critically, "zero-day" exploits. These zero-day attacks target previously unknown software vulnerabilities, meaning no patch or pre-existing defense mechanism exists for them. This escalating sophistication of cyber-attacks mandates a fundamental paradigm shift away from static, reactive security postures, which are perpetually one step behind the attackers. The new imperative is the development and deployment of dynamic, proactive, predictive, and intelligent defense mechanisms that can adapt in real-time. Network Intrusion Detection Systems (NIDS) represent a critical and indispensable layer within any modern, in-depth cybersecurity architecture. Functioning as a vigilant and intelligent sentinel, a NIDS operates as a crucial second line of defense, complementing perimeter security controls such as firewalls. While a firewall acts as a "locked door," blocking traffic from known bad sources, a NIDS acts as a "security guard" actively monitoring all inbound and outbound network traffic for suspicious patterns and malicious activities that may have successfully bypassed these initial static defenses. However, the efficacy of traditional signature-based NIDS is fundamentally constrained. These systems operate by maintaining a massive database of known attack "signatures"—unique patterns of data or packet sequences associated with known malware or exploits. When network traffic matches a pattern in this database, an alert is raised. While this method is extremely fast and accurate for known threats, its core limitation is its foundational reliance on this pre-existing database. By definition, a signature-based NIDS

is structurally incapable of identifying novel, previously unseen attack vectors, including all zero-day exploits. This creates a critical, high-risk security gap, leaving networks defenseless against the most advanced and damaging categories of modern cyber threats. To address this gap, anomaly-based detection has emerged as a more powerful and adaptive alternative. This paradigm involves building a robust, statistical baseline model of "normal" network behavior. Any network traffic that significantly deviates from this established baseline is flagged as a potential anomaly, and thus a potential intrusion. This is where Machine Learning (ML) becomes an exceptionally suitable and powerful tool, as it can autonomously learn the complex, multi-dimensional patterns of normal behavior from vast datasets, a task that is impossible for humans to perform manually.

Objective:

This project undertakes a comprehensive, systematic, and empirical investigation into the application and efficacy of a suite of classical machine learning algorithms for the development of a high-performance, anomaly-based Network Intrusion Detection System. The primary objective is to move beyond mere theoretical postulation and to practically implement, rigorously evaluate, and systematically compare the performance of three distinct paradigms of supervised machine learning classifiers. The models selected for this comparative study are: K-Nearest Neighbors (KNN), which serves as a benchmark for instance-based, non-parametric learning; Linear Discriminant Analysis (LDA), which represents a classical, generative statistical approach effective for class separation; and the Support Vector Machine (SVM), which represents a powerful margin-based classification method renowned for its effectiveness in high-dimensional feature spaces. A central objective is to benchmark the performance of these disparate models on the widely recognized and academically validated NSL-KDD dataset. The use of this standardized dataset ensures that the project's findings are reproducible, verifiable, and can be accurately compared against the existing body of NIDS research.

Furthermore, a key deliverable of this project is to bridge the significant gap between abstract academic research and practical, operational deployment. This is achieved by translating the empirical findings into a tangible, high-fidelity, and interactive web dashboard. This proof-of-concept tool is designed to demonstrate how a trained and serialized machine learning model can be operationalized to provide actionable intelligence for security analysts and network administrators.

Methodology:

The project adheres to a structured, disciplined, and multi-stage data science pipeline, ensuring that the process is methodical, transparent, and reproducible from initial data ingestion to final model evaluation.

The process commences with Data Acquisition and Analysis, involving the procurement of the NSL-KDD dataset. This dataset is a refined version of the original KDD Cup '99 dataset, specifically corrected to remove the redundant records that previously skewed model evaluations. The dataset provides 41 features for each network connection, which are categorized into Basic, Content, and Traffic-based features, and labels are grouped into four main attack categories: DoS, Probe, R2L, and U2R.

The next critical phase is Data Preprocessing and Feature Engineering. This is a multi-step process to transform the raw data into a format suitable for machine learning. First, the problem is framed as a binary classification task, consolidating all 22 unique attack types into a single "Attack" class (labeled as 1) and the "normal" class (labeled as 0). Second, One-Hot Encoding is applied to the three non-numeric, categorical features (protocol_type, service, flag). This technique creates new binary columns for each category, preventing the models from learning a false ordinal relationship. Third, Min-Max Normalization is applied to all numerical features. This scales all values to a uniform range of [0, 1], a crucial step to ensure that distance-based algorithms like KNN and SVM are not disproportionately influenced by features with large magnitudes (e.g., src_bytes).

Following preprocessing, the Model Training phase is executed. The three classifiers (KNN, LDA, SVM) are trained *only* on the fully preprocessed KDDTrain+.txt dataset.

The final methodological step is Model Evaluation. To obtain an unbiased assessment of their generalization performance, the trained models are rigorously evaluated against the entirely unseen and separately preprocessed KDDTest+.txt dataset. This strict separation of training and testing data is critical to prevent data leakage and ensure the results reflect real-world performance.

Implementation:

Transcending a purely theoretical evaluation, this project's implementation is bifurcated into two distinct components: an offline training pipeline and a live prediction dashboard.

The Offline Pipeline (train_and_evaluate.py) is an automated script that orchestrates the entire model development lifecycle. It loads and preprocesses both the train and test data, initializes the three ML models, and then systematically trains and evaluates each one. Upon completion, it saves the comparative performance metrics (Accuracy, Precision, Recall, F1-Score) to a model_performance.csv file. Finally, it identifies the best-performing model (KNN, in this case), serializes it as nids_model.pkl using joblib, and also saves the fitted scaler.pkl and the model_columns.pkl list. These three serialized artifacts are the essential "assets" for the live application.

The Live Application Dashboard (app.py) is a practical, user-facing tool built using the Streamlit framework. On startup, the application loads the three saved assets. The UI is designed with a two-column layout that only appears *after* a file is uploaded, maintaining a clean initial state. The user can

upload one or more CSV files containing network traffic. The application then iterates through each file, creating a separate expandable section for its analysis.

Inside each file's analysis section, the application performs a real-time preprocessing pipeline: it applies one-hot encoding, aligns the file's columns to the *loaded* model_columns.pkl, and transforms the numerical data using the *loaded* scaler.pkl. This guarantees that the new data is processed identically to the training data. The processed data is then fed into the *loaded* nids_model.pkl to generate immediate predictions.

The results are presented in a highly intuitive format: a high-level Prediction Summary uses st.metric to show "Total Connections Analyzed," "Attacks Found," and "Normal Connections," along with a prominent st.warning ("🚨 Attack Detected!") or st.success ("✅ No Attacks Detected."). Below this, two tabs, "Detailed Results" and "Input Data," allow the user to inspect the row-by-row predictions (with attacks highlighted) or their original uploaded data.

Conclusion:

The empirical results obtained from this project unequivocally demonstrate the significant potential and high efficacy of applying classical machine learning techniques for the construction of robust and effective anomaly-based Network Intrusion Detection Systems. The project provides quantitative, reproducible evidence that data-driven approaches can successfully model the complex, high-dimensional behavior of network traffic and accurately distinguish between benign and malicious activities.

The detailed comparative analysis revealed that while all three models performed well, the K-Nearest Neighbors (KNN) model yielded the best overall performance, achieving the highest Accuracy (0.7694) and F1-Score (0.7619), making it the most balanced classifier for this task. The Linear Discriminant Analysis (LDA) model also proved to be a strong contender, achieving the highest Precision (0.9249), making it the most reliable at avoiding false positives.

This project's primary contribution is the successful bridging of the gap between theoretical academic research and practical, operational deployment. It delivers not just a research paper but a fully functional, end-to-end system that operationalizes a trained model into an interactive, user-friendly, and multi-file analysis dashboard.

The project concludes by identifying a clear roadmap for Future Work. This includes integrating more advanced Deep Learning models (such as LSTMs for time-series analysis and Autoencoders for superior anomaly detection), re-evaluating the entire pipeline on more modern datasets (like CIC-IDS2017) to ensure contemporary relevance, and, most importantly, transitioning the system from a batch-file processor to a real-time detection pipeline capable of live packet capture and analysis.

I. INTRODUCTION

A. Background and Motivation

- The Proliferation of Digital Connectivity and the Modern Network Infrastructure:

- The contemporary era is defined by an unprecedented and accelerating proliferation of digital connectivity, a phenomenon that has fundamentally reshaped global commerce, communication, and societal structures.
 - Networks are no longer peripheral support systems; they have evolved into the central nervous system of modern infrastructure, underpinning critical sectors ranging from finance and healthcare to energy and transportation.
 - The advent of the Internet of Things (IoT), cloud computing, and ubiquitous mobile access has exponentially increased the number of interconnected devices, creating an immeasurably vast and complex attack surface.
 - This hyper-connectivity, while a catalyst for innovation and efficiency, has concurrently exposed these critical infrastructures to a continuously evolving and increasingly sophisticated landscape of cyber threats, making network security a paramount concern for global stability and progress.
 - The Significance and Impact of Cyber Threats:
 - The risks associated with this digital dependency are profound and multifaceted. Unauthorized access to sensitive networks can lead to catastrophic data breaches, resulting in the theft of intellectual property, personal identifiable information (PII), and confidential corporate data.
 - Service disruptions, often orchestrated through Denial of Service (DoS) attacks, can cripple essential services, leading to significant financial losses, reputational damage, and, in some cases, threats to public safety.
 - The economic impact of cybercrime is staggering, with global costs estimated in the trillions of dollars annually, encompassing not only direct financial theft but also the costs of remediation, regulatory fines, and loss of customer trust.
 - These escalating risks necessitate a transition from a passive security posture to a proactive, intelligent, and resilient framework, making robust and adaptive network security an indispensable strategic requirement for organizations and nation-states alike.
 - The Role of Network Intrusion Detection Systems (NIDS) in a Layered Security Architecture:
 - No single security measure is infallible. Therefore, modern cybersecurity relies on a "defense-in-depth" or layered security strategy, where multiple, overlapping security controls are implemented to protect assets.
 - Network Intrusion Detection Systems (NIDS) are a critical and non-negotiable component of this layered strategy. They function as a vigilant surveillance mechanism, operating behind preventative perimeter controls like firewalls and access control lists.
 - The primary function of a NIDS is to continuously monitor and analyze network traffic in real-time to identify malicious activities, policy violations, or anomalous behaviors that may have successfully bypassed these initial lines of defense.
 - By providing timely and actionable alerts, NIDS empower security analysts to respond swiftly to potential security incidents, thereby mitigating damage, containing threats, and facilitating post-incident forensic analysis.
- B. Problem Statement
- The Inherent Limitations of Traditional Signature-Based Detection:
 - The predominant methodology employed by traditional Network Intrusion Detection Systems is signature-based detection. This approach operates analogously to antivirus software, where incoming network packets are meticulously inspected and compared against a vast, predefined database of known attack patterns, or "signatures."
 - While this method is highly effective and computationally efficient at identifying well-known and documented threats, its efficacy is fundamentally predicated on the existence of a prior signature for an attack.
 - The fundamental and critical limitation of this approach is its inherent inability to detect novel, polymorphic, or sophisticated "zero-day" attacks. These are attacks that exploit previously unknown vulnerabilities, and for which no signature has yet been developed or distributed.
 - In an era where attackers can rapidly generate new malware variants and exploit chains, this reliance on historical knowledge creates a significant and dangerous security gap, leaving networks vulnerable during the critical period between the discovery of a new threat and the deployment of its corresponding signature.
 - The Promise and Dynamics of Anomaly-Based Detection:
 - To address the shortcomings of signature-based systems, anomaly-based detection offers a more dynamic, adaptive, and forward-looking alternative.

- Instead of relying on a blacklist of known bad patterns, this paradigm involves creating a robust, statistical baseline model of "normal" or legitimate network behavior. This baseline comprehensively characterizes typical traffic patterns, protocols, and data flows within the network.
- The core principle is that any significant deviation from this established baseline of normalcy is flagged as a potential anomaly or intrusion. This approach is, by its nature, capable of detecting novel attacks without any prior knowledge of their specific characteristics.
- It effectively shifts the detection focus from identifying known threats to identifying abnormal behavior, a far more powerful and future-proof strategy against the evolving threat landscape.
- The Synergy Between Machine Learning and Anomaly-Based NIDS:
 - Machine Learning (ML) is an exceptionally well-suited and powerful technology for the construction of high-fidelity, anomaly-based NIDS.
 - ML algorithms possess the intrinsic ability to autonomously learn complex, non-linear patterns and subtle correlations from vast, high-dimensional datasets without requiring explicit, rule-based programming. This is particularly advantageous for analyzing network traffic, which is characterized by its high volume, velocity, and complexity.
 - By training on large corpuses of network data, ML models can independently build the baseline of normal behavior and subsequently identify deviations with a high degree of sensitivity and accuracy.
 - The core problem that this project seeks to address is therefore the systematic and empirical evaluation and comparison of several well-established, classical machine learning algorithms to scientifically determine their suitability, performance trade-offs, and overall effectiveness for building a high-performance, anomaly-based NIDS.
- This includes the implementation of robust feature engineering techniques such as one-hot encoding for categorical variables to prevent model bias, and Min-Max normalization for numerical features to ensure equitable feature contribution, thereby laying a solid and reproducible foundation for the modeling phase.
- Rigorous Empirical Model Comparison:
 - The project provides a rigorous, scientifically grounded, side-by-side performance evaluation of three distinct and widely-used paradigms of supervised classification algorithms: the instance-based K-Nearest Neighbors (KNN), the statistical Linear Discriminant Analysis (LDA), and the margin-based Support Vector Machine (SVM).
 - This empirical comparison is conducted using a standardized, unseen test set and a comprehensive suite of performance metrics (Accuracy, Precision, Recall, F1-Score), ensuring that the conclusions drawn are objective, quantifiable, and directly comparable.
- Practical Implementation and Bridging the Research-Application Gap:
 - A significant contribution of this project is the translation of the theoretical analysis into a tangible, deployable, and user-friendly tool.
 - The development of the interactive Streamlit dashboard serves as a powerful proof-of-concept, demonstrating the entire end-to-end pipeline of how a trained machine learning model can be successfully operationalized and deployed for practical, real-world use by security analysts or network administrators.
- Commitment to Open and Reproducible Science:
 - The entire project, from the initial stages of data acquisition and exploration to the final application deployment and code implementation, is thoroughly and meticulously documented.
 - This commitment to detailed documentation ensures maximum clarity, facilitates the reproducibility of the experimental results by other researchers, and enhances the educational value of the project for students and practitioners in the field of cybersecurity and machine learning.

C. Project Contributions

- Comprehensive and Replicable Data Preprocessing:
 - This project delivers a meticulously designed and thoroughly documented data preprocessing pipeline, specifically tailored for the unique characteristics of the NSL-KDD dataset.

D. Paper Structure

- Section II: Literature Survey
 - This section will provide a comprehensive review of the existing body of academic and industry literature concerning the application of

machine learning and data mining techniques in the domain of network security and intrusion detection, establishing the context and foundation for this work.

- Section III: Methodology and System Architecture

- This section will meticulously describe the complete methodology employed in the project. It will include a detailed overview of the benchmark NSL-KDD dataset, a step-by-step breakdown of the data preprocessing pipeline, and an exposition of the theoretical foundations and operational mechanics of the machine learning models utilized.

- Section IV: Implementation Details

- This section will provide an in-depth account of the practical implementation of the project. It will detail the development of the Python-based model training and evaluation pipeline using Scikit-learn and the construction of the interactive front-end dashboard using the Streamlit framework.

- Section V: Results and Discussion

- This section will present and critically discuss the empirical results obtained from the model evaluation phase. It will feature a detailed comparative analysis of the performances of the KNN, LDA, and SVM models, complete with visualizations and a discussion of the performance trade-offs.

- Section VI: Conclusion and Future Work

- This final section will conclude the paper by summarizing the key findings and contributions of the project. It will also outline a strategic and promising roadmap for future research and development, including potential enhancements and new avenues of investigation.

II. LITERATURE SURVEY

A. Evolution of Intrusion Detection

- Foundations in Rule-Based Expert Systems:

- Early academic and commercial research in the field of intrusion detection, dating back several decades, was predominantly rooted in the paradigm of expert systems. These initial systems were meticulously handcrafted, rule-based engines.
- Security experts would manually codify their domain knowledge into a set of explicit IF-THEN rules. For instance, a rule might state: IF a single IP address makes more than 100 failed login

attempts in one minute, THEN flag it as a potential brute-force attack.

- The core of these systems was a knowledge base containing hundreds or thousands of such rules, which were manually curated and updated. This process was extremely labor-intensive, time-consuming, and highly dependent on the continuous availability of specialized human expertise.
- While effective for known threat vectors, these rule-based systems were inherently brittle. They struggled to adapt to new or slightly modified attack techniques and were prone to generating a high volume of false positives if the rules were not perfectly calibrated to the specific network environment.

- The Paradigm Shift to Data-Driven Methodologies:

- The transition from purely manual, rule-based systems to more automated, data-driven approaches marked a significant inflection point in the evolution of NIDS. This shift was catalyzed by two primary technological advancements.
- First, the increasing availability of large-scale, labeled network traffic datasets provided the raw material necessary for training and validating statistical and machine learning models. The creation of public repositories for these datasets fostered a competitive and collaborative research environment.
- Second, the exponential growth in computational power, as described by Moore's Law, made it feasible to apply complex, computationally intensive algorithms to these massive datasets. Tasks that were once computationally prohibitive, such as training complex classifiers or clustering terabytes of traffic data, became achievable.
- This transition effectively moved the field from a model of "knowledge engineering," where human expertise was manually encoded, to a model of "machine learning," where the system could autonomously learn the patterns of normal and malicious behavior directly from the data itself.

B. Benchmark Datasets in NIDS Research

- The KDD Cup 1999 Dataset: A Foundational Landmark:

- The KDD Cup 1999 dataset is widely regarded as a foundational landmark in the history of NIDS research. It was one of the first comprehensive, publicly available datasets specifically created for the evaluation of intrusion detection systems.
- Its release provided a crucial common ground and a standardized benchmark, allowing researchers from around the world to objectively compare the performance of their diverse algorithms on a level playing field for the first time.
- However, subsequent in-depth analyses revealed significant flaws within the dataset. The most severe criticism was the presence of a massive number of redundant records. Approximately 78% of the

records in the training set and 75% in the test set were duplicates.

- This redundancy had a detrimental effect on model evaluation, as it could severely bias classifiers. A model might achieve an artificially high accuracy simply by learning to favor the more frequent, redundant attack types, while performing poorly on rarer but more critical attacks like U2R or R2L.
- The NSL-KDD Dataset: A Refined and More Realistic Benchmark:
 - The NSL-KDD dataset was meticulously crafted and introduced as a direct response to the identified limitations of its predecessor. It is not a new dataset from scratch but rather an improved, refined version of KDD'99.
 - The primary improvement was the complete removal of duplicate records from both the training and testing sets. This crucial step ensures that the models are evaluated on a more diverse and non-redundant set of instances, preventing the aforementioned bias.
 - By removing the redundant records, the NSL-KDD dataset provides a more balanced and realistic class distribution. This makes the evaluation task more challenging and a better reflection of a real-world scenario, where a classifier must be effective against a variety of attack types, not just the most common ones.
 - Due to these significant improvements, the NSL-KDD dataset has become a de facto standard benchmark in contemporary NIDS research. It remains a widely used and respected dataset for evaluating and comparing the performance of new and existing machine learning algorithms for intrusion detection.
- The Emergence of Modern Datasets for Contemporary Threats:
 - While NSL-KDD remains a valuable benchmark, the nature of network traffic and cyber attacks has evolved dramatically since its creation. Recognizing this, researchers have developed more recent and sophisticated datasets.
 - The CIC-IDS2017 dataset, for example, was designed to capture more realistic and modern network traffic patterns. It includes traffic generated from a variety of contemporary attack vectors, such as Brute Force FTP, Brute Force SSH, DoS, Heartbleed, Web Attacks (including SQL Injection and XSS), and botnet infiltration.
 - Similarly, the UNSW-NB15 dataset was created to address the need for a more comprehensive benchmark that includes a wider variety of modern, low-footprint attack types. It contains a hybrid of real-world normal network traffic and synthetically generated attack traffic, encompassing nine distinct families of attacks.
 - These newer datasets are crucial for the future of NIDS research, as they provide a more challenging and accurate environment for testing the ability of

machine learning models to cope with the complexity and subtlety of today's cyber threats.

C. Application of Machine Learning in NIDS

- Broad Spectrum of Algorithmic Exploration:
 - The application of machine learning to intrusion detection is a rich and diverse field of study. A vast number of academic papers and studies have explored the efficacy of a wide array of ML algorithms for this task.
 - These studies span the full spectrum of machine learning paradigms, including supervised learning (where models are trained on labeled data), unsupervised learning (where models must find patterns in unlabeled data, ideal for anomaly detection), and semi-supervised learning (which uses a combination of labeled and unlabeled data).
 - The choice of algorithm often depends on the specific goals of the NIDS, such as whether it is designed for a specific type of network, the computational resources available, and the desired trade-off between detection accuracy and false alarm rates.
- K-Nearest Neighbors (KNN): The Instance-Based Approach:
 - The K-Nearest Neighbors (KNN) algorithm has been frequently noted in the literature for its conceptual simplicity and surprisingly high effectiveness in NIDS contexts.
 - Numerous studies applying KNN to the NSL-KDD dataset have reported high accuracy rates, demonstrating its capability to effectively classify network connections in a high-dimensional feature space.
 - However, a significant and consistently cited drawback of KNN is its computational cost during the prediction phase. As a "lazy learner," KNN performs no upfront model building. Instead, it must compute the distance from a new data point to every single point in the training dataset to make a prediction. For large datasets, this can make real-time prediction prohibitively slow and resource-intensive.
- Support Vector Machines (SVM): The Margin-Based Classifier:
 - Support Vector Machines (SVM) are a class of powerful supervised learning models that are frequently praised in NIDS literature for their strong theoretical foundations and excellent empirical performance.
 - Their ability to find an optimal decision boundary (hyperplane) that maximizes the margin between different classes makes them particularly robust, especially in high-dimensional feature spaces like those found in network traffic data with dozens of features.
 - Research has consistently shown SVM to be a strong and reliable performer in binary classification tasks for NIDS (i.e., distinguishing 'Normal' from 'Attack').

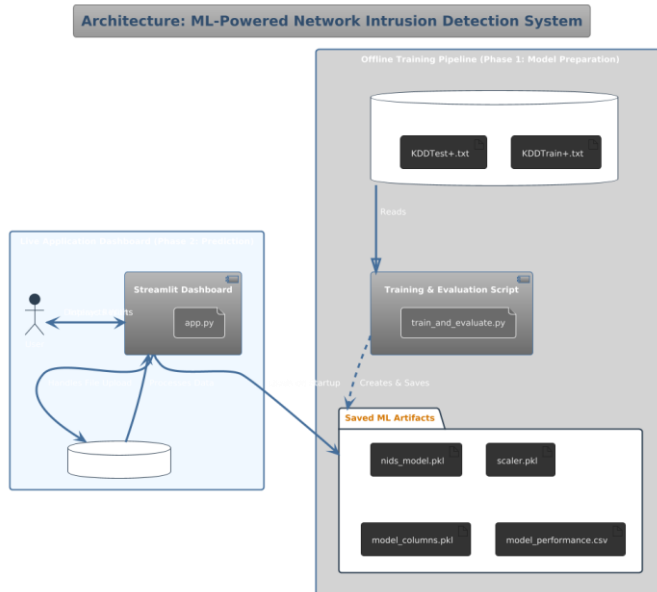
- The power of SVMs can be further enhanced through the use of the "kernel trick," which allows them to model complex, non-linear relationships in the data, although this project focused on a computationally efficient linear kernel.
- Linear Discriminant Analysis (LDA): The Statistical Approach:
 - Linear Discriminant Analysis (LDA) is a versatile statistical technique that serves a dual purpose. While it is primarily known as a feature extraction and dimensionality reduction technique, it can also be directly used as a powerful and computationally efficient classifier.
 - Studies have shown LDA to be an effective method for NIDS, often providing performance comparable to more complex models but with significantly lower training and prediction times.
 - However, the effectiveness of LDA is predicated on several underlying statistical assumptions, namely that the data for each class follows a Gaussian (normal) distribution and that all classes share the same covariance matrix.
 - This assumption of a linear relationship between features may not always hold true for the intricate and often non-linear patterns present in complex network attack data, which can be a potential limitation of this approach.

III. METHODOLOGY AND SYSTEM ARCHITECTURE

A. System Workflow

- Adherence to a Structured Methodological Framework:
 - The project's execution is underpinned by a structured and systematic workflow, mirroring established best practices in data science and machine learning project management².
 - This adherence to a formal process ensures that each phase of the project is conducted with rigor, that the results are reproducible, and that the overall architecture is logical and transparent³.
 - The workflow is designed as a sequential pipeline, where the output of each stage serves as the validated input for the subsequent stage, minimizing errors and ensuring the integrity of the final deployed system⁴.
 - The architecture is visually depicted in a system workflow diagram to provide a clear, high-level overview of the end-to-end process⁵.
- Phase 1: Data Acquisition:
 - The foundational phase of the project involves the procurement of a suitable dataset for training and evaluating the intrusion detection models⁶.
 - After a thorough review of available benchmark datasets, the NSL-KDD dataset is selected⁷.
 - This choice is deliberate, based on its established reputation in the academic community and its specific design to overcome the statistical limitations of the earlier KDD Cup 1999 dataset⁸.
 - The dataset is procured from a trusted academic repository, ensuring its integrity and authenticity⁹.
 - Both the designated training set (KDDTrain+.txt) and the separate, unseen testing set (KDDTest+.txt) are acquired to facilitate a robust and unbiased evaluation of model performance¹⁰.
- Phase 2: Data Preprocessing:
 - This phase is arguably the most critical, as the quality and format of the data directly impact the performance and reliability of any machine learning model¹¹.
 - The raw, unprocessed data from the NSL-KDD dataset is subjected to a comprehensive, multi-step pipeline of cleaning, transformation, and normalization¹².
 - This includes the meticulous handling of categorical data through one-hot encoding, the conversion of the multi-class problem into a binary classification problem through label encoding, and the scaling of all numerical features to a uniform range using Min-Max normalization¹³.
 - Each of these steps is detailed further in Section III-C¹⁴.
- Phase 3: Model Training:
 - In this phase, the fully preprocessed, clean, and normalized training data is used to train the selected machine learning models¹⁵.
 - Three distinct supervised learning algorithms are implemented using the Scikit-learn library in Python: K-Nearest Neighbors (KNN), Linear Discriminant Analysis (LDA), and the Support Vector Machine (SVM)¹⁶.
 - Each model is trained independently on the entire preprocessed training dataset, allowing each algorithm to learn the underlying patterns and decision boundaries that differentiate between 'Normal' and 'Attack' network traffic¹⁷.
- Phase 4: Model Evaluation:
 - To assess the generalization capability and real-world effectiveness of the trained models, they are subjected to a rigorous evaluation phase¹⁸.
 - The models are evaluated on the preprocessed unseen test data from KDDTest+.txt¹⁹.
 - Using a separate and unseen test set is crucial to prevent data leakage and obtain an unbiased estimate of the models' performance on new data²⁰.
 - A suite of standard, quantitative performance metrics is employed, including Accuracy, Precision, Recall, and F1-Score²¹.
 - This multi-metric approach provides a holistic view of each model's strengths and weaknesses, moving beyond a simple accuracy score²².
- Phase 5: Deployment:
 - The final phase bridges the gap between theoretical modeling and practical application²³.
 - Based on the results of the evaluation phase, the best-performing model is identified (in this case, the one with the highest accuracy and F1-score)²⁴.

- This champion model, along with its corresponding data scaler and the list of feature columns, is serialized and saved to disk using the joblib library²⁵.
- These saved assets are then integrated into a fully interactive and user-friendly web application, developed using the Streamlit framework, thereby deploying the model for on-demand analysis and prediction²⁶.



B. Dataset Description: NSL-KDD

- Origin and Purpose:
 - The NSL-KDD dataset is a refined and improved version of the seminal KDD Cup 1999 dataset, which was a pioneering resource in the field of intrusion detection research²⁸.
 - It was specifically created to address and rectify the statistical anomalies present in the KDD'99 dataset, most notably the presence of a vast number of redundant records which tended to bias classifiers towards more frequent attack types.
- Structure and Contents:
 - The dataset is composed of a large collection of individual network connection records, where each record represents a sequence of TCP packets over a period of time, flowing between a source and destination IP address under a well-defined protocol³⁰.
 - Each of these connection records is described by a vector of 41 features, providing a rich, multi-dimensional representation of the connection's behavior³¹.
 - The dataset is pre-partitioned into a training set (KDDTrain+.txt) and a distinct test set (KDDTest+.txt), facilitating standardized and comparable evaluations across different research studies³².
- Feature Categorization:
 - The 41 features are logically categorized into three distinct groups based on their nature and how they are derived³³:

- Basic Features (9 features): These are the intrinsic, fundamental attributes of an individual TCP connection itself, derived directly from the packet headers without any deeper inspection of the payload³⁴. This category includes features like the duration of the connection, the protocol_type (e.g., TCP, UDP, ICMP), the destination network service (e.g., HTTP, FTP), and the connection status flag (e.g., SF, REJ)³⁵.
- Content Features (13 features): These features require a deeper inspection of the TCP packet's payload and are based on domain knowledge of network protocols and attack methodologies³⁶. This category includes features such as num_failed_logins, logged_in, and root_shell³⁷.
- Traffic Features (19 features): These features are computed statistically over a window of recent connections, providing a temporal context that is crucial for detecting certain types of attacks³⁸. They are designed to capture patterns that emerge over time³⁹. This category includes "same-host" features (e.g., same_srv_rate) and "same-service" features⁴⁰.

○ Classification Labels:

- Each record in the dataset is meticulously labeled, falling into one of two main categories: 'Normal' for legitimate traffic, or a specific attack type⁴¹.
- The dataset includes a diverse set of 22 different attack types, which are further aggregated into four broad, well-known categories of network attacks⁴²:
- Denial of Service (DoS): Attacks designed to overwhelm a target system's resources, rendering it unable to respond to legitimate requests⁴³. Examples include smurf, neptune, and teardrop⁴⁴.
- Probe (Probing): Attacks where an adversary systematically scans a network to gather information about potential vulnerabilities, active hosts, and open ports⁴⁵. Examples include portsweep and nmap⁴⁶.
- Remote to Local (R2L): Attacks in which an attacker, operating from a remote machine, exploits a vulnerability to gain unauthorized local user access on a target machine⁴⁷. Examples include ftp_write and guess_passwd⁴⁸.
- User to Root (U2R): Attacks where an attacker has already gained local user-level access on a system and subsequently exploits a vulnerability to escalate their privileges to the root (administrator) level⁴⁹. Examples include loadmodule and perl⁵⁰.

C. Data Preprocessing Pipeline

- The Imperative of Preprocessing:
 - Raw network traffic data, as captured in datasets like NSL-KDD, is inherently heterogeneous and unstructured⁵¹.
 - It contains a mix of numerical and categorical data, features with vastly different scales, and labels that need to be transformed for specific modeling tasks⁵².
 - Directly feeding this raw data into machine learning models would lead to poor performance,

model bias, and potential computational errors⁵³. Therefore, a robust, multi-step preprocessing pipeline is an essential and non-negotiable prerequisite⁵⁴.

- Step 1: Feature Encoding (Transformation of Categorical Data):
 - The NSL-KDD dataset contains three critical features that are non-numeric: `protocol_type`, `service`, and `flag`⁵⁵.
 - Since machine learning algorithms are mathematical models that operate on numerical tensors, these categorical features must be converted into a numerical representation⁵⁶.
 - A naive approach like integer encoding (e.g., `tcp=0`, `udp=1`, `icmp=2`) would impose an artificial ordinal relationship that does not exist, implying that 'udp' is somehow "greater" than 'tcp'⁵⁷.
 - To circumvent this, one-hot encoding was employed. This technique transforms each categorical feature into a set of new binary (0 or 1) features, one for each unique category⁵⁸. For example, `protocol_type` would be expanded into three new features: `protocol_type_tcp`, `protocol_type_udp`, and `protocol_type_icmp`⁵⁹. This creates a sparse but accurate numerical representation without implying any false order⁶⁰.
- Step 2: Label Encoding (Problem Framing):
 - The target variable, the 'class' column, is multi-class, containing the label 'normal' and 22 distinct attack labels⁶¹.
 - For the scope of this project, the primary objective was to build a system that can effectively distinguish any malicious activity from normal behavior⁶².
 - Therefore, the problem was framed as a binary classification task⁶³.
 - This was achieved through a straightforward label encoding process: the 'normal' label was mapped to the integer 0, and all 22 attack labels were consolidated and mapped to the integer 1⁶⁴. This transformation simplifies the problem and allows for the use of standard binary classification models and metrics⁶⁵.
- Step 3: Data Normalization (Feature Scaling):
 - An examination of the numerical features in the dataset reveals that they exist on vastly different scales and ranges⁶⁶. For instance, the `src_bytes` feature can have values in the millions, while rate-based features like `error_rate` are bounded between 0 and 1⁶⁷.
 - This discrepancy in scales can severely degrade the performance of machine learning algorithms, especially those that rely on distance calculations (like KNN) or gradient-based optimization (like SVM)⁶⁸. Features with larger magnitudes can disproportionately dominate the learning process⁶⁹.
 - To address this, Min-Max Scaling was applied to all numerical features⁷⁰. This technique linearly transforms each feature to a common range, typically [0, 1]⁷¹.

- The transformation is performed using the formula:
$$X_{\text{scaled}} = (X - X_{\min}) / (X_{\max} - X_{\min})$$
⁷².
- This crucial step ensures that every feature contributes equally to the model's decision-making process, leading to a more robust, stable, and accurate model⁷³.

D. Machine Learning Models

- 1) K-Nearest Neighbors (KNN):
 - Fundamental Principle: KNN is a non-parametric, supervised learning algorithm that is fundamentally based on the principle of feature similarity⁷⁴. It is one of the simplest yet surprisingly effective classification algorithms⁷⁵.
 - Operational Mechanism: Unlike many other algorithms, KNN does not learn an explicit discriminative function from the training data⁷⁶. Instead, it memorizes the entire training dataset⁷⁷. When it needs to classify a new, unseen data point, it calculates the distance between this point and every single point in the training data, identifying the 'k' closest training instances (the "nearest neighbors")⁷⁸.
 - Classification Process: The classification of the new data point is determined by a majority vote among its 'k' identified neighbors⁷⁹. For instance, if $k=5$ and three of the five nearest neighbors belong to the 'Attack' class, the new point will also be classified as 'Attack'⁸⁰.
 - Core Component - Distance Metric: The notion of "closeness" or "similarity" is quantified using a distance metric⁸¹. The most common choice, and the one used in this project, is the Euclidean distance, which is the straight-line distance between two points in the multi-dimensional feature space⁸².
 - Critical Hyperparameter 'k': The choice of 'k', the number of neighbors to consider, is a critical hyperparameter that can significantly affect the model's performance⁸³. A small 'k' can make the model sensitive to noise, while a large 'k' can be computationally expensive⁸⁴. For this project, a conventional and widely-used value of $k=5$ was selected⁸⁵.
 - Algorithmic Nature: KNN is often referred to as a "lazy learner" because it performs no computation during the training phase; it simply stores the data⁸⁶. All the intensive work happens at prediction time, which can be a significant performance bottleneck in real-time systems⁸⁷.
- 2) Linear Discriminant Analysis (LDA):
 - Fundamental Principle: LDA is a classical statistical method that serves as both a classifier and a dimensionality reduction technique⁸⁸. It is a generative model, meaning it models the distribution of the data for each class⁸⁹.
 - Operational Mechanism: The primary goal of LDA is to find a lower-dimensional feature space and a projection of the data onto this space that maximizes the separability between the different

classes⁹⁰. It achieves this by finding a projection that maximizes the ratio of the between-class variance (distance between class means) to the within-class variance (spread of data within a class)⁹¹.

- Underlying Assumptions: The mathematical foundation of LDA rests on two key assumptions: first, that the data for each class is drawn from a multivariate Gaussian (normal) distribution, and second, that all classes share the same covariance matrix⁹². While these assumptions may not hold perfectly for real-world data, LDA often performs remarkably well⁹³.
- Classification Process: Once the optimal projection (the linear discriminant) is found, LDA can classify new data points by projecting them into the new feature space and assigning them to the class whose mean is closest⁹⁴. This makes it an efficient and powerful linear classifier⁹⁵.
- 3) Support Vector Machine (SVM):
 - Fundamental Principle: The Support Vector Machine is a highly effective and widely used supervised learning algorithm known for its strong theoretical guarantees and excellent empirical performance, particularly on high-dimensional data⁹⁶.
 - Operational Mechanism: The central idea of SVM is to find an optimal hyperplane that separates the data points of different classes in the feature space⁹⁷. A hyperplane is a decision boundary; for a two-dimensional space, it is a line, and for a three-dimensional space, it is a plane⁹⁸.
 - The Concept of the Optimal Hyperplane: While there might be many hyperplanes that can separate the data, the "optimal" hyperplane is the one that has the largest margin⁹⁹. The margin is defined as the distance between the hyperplane and the nearest data points from either class¹⁰⁰. A larger margin generally leads to better generalization performance on unseen data¹⁰¹.
 - The Role of Support Vectors: The specific training data points that are closest to the optimal hyperplane are called the support vectors¹⁰². These are the most critical data points in the dataset because they are the ones that define the position and orientation of the hyperplane¹⁰³. All other data points are irrelevant to the final model¹⁰⁴.
 - The Kernel Trick: For this project, a linear kernel was employed, which means the model attempts to find a linear decision boundary¹⁰⁵. This is computationally efficient and works well when the data is mostly linearly separable¹⁰⁶. More complex, non-linear relationships can be modeled by SVMs through the use of other kernels, such as the Polynomial or Radial Basis Function (RBF) kernel¹⁰⁷.

IV. IMPLEMENTATION

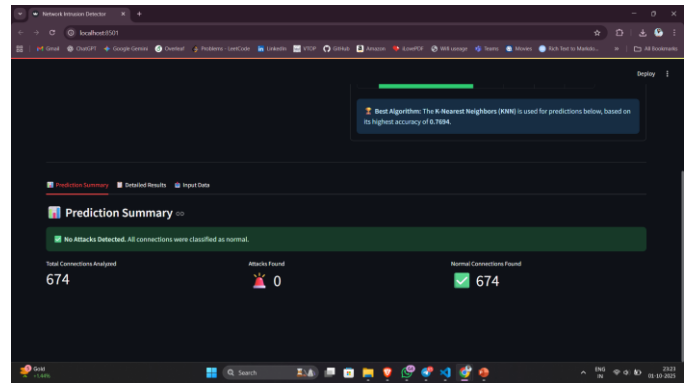
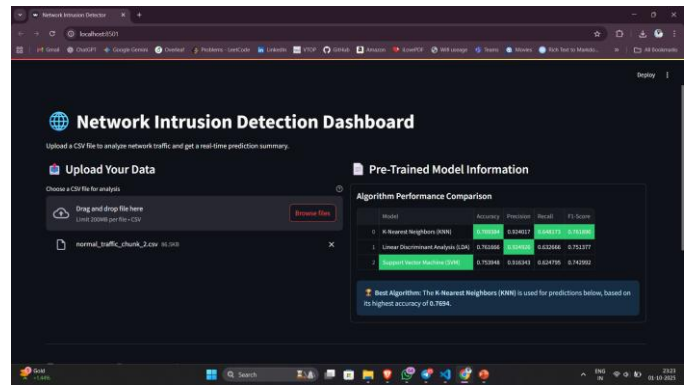
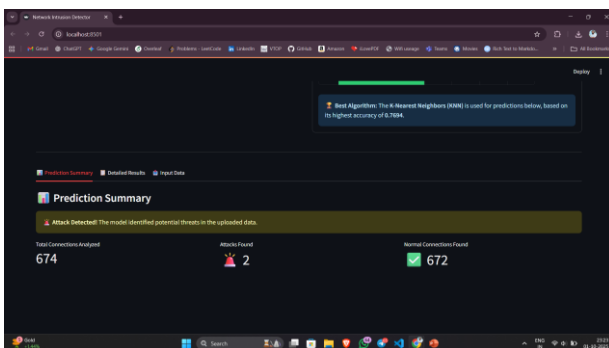
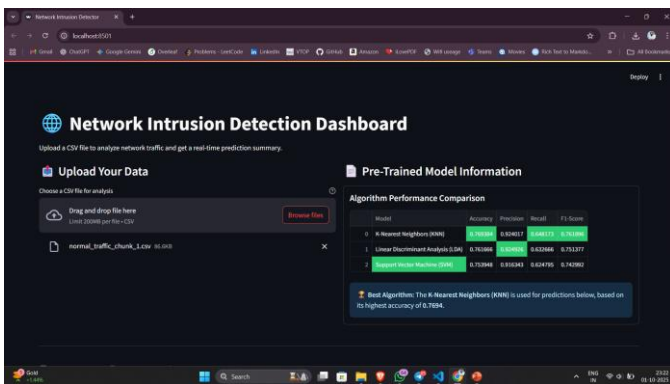
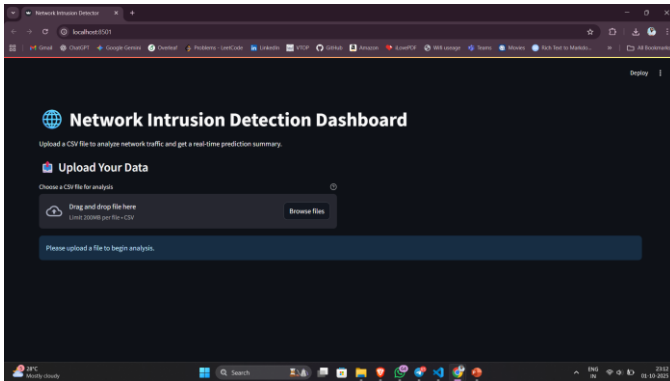
A. Model Training and Evaluation Pipeline (train_and_evaluate.py)

- Purpose:
 - This Python script serves as the core engine for training, evaluating, and saving the models. It is an automated, offline process that prepares all assets for the live application.
- Execution Flow:
 - The script begins by loading the KDDTrain+.txt and KDDTest+.txt datasets using the Pandas library.
 - It executes the full data preprocessing pipeline as described in Section III-C.
 - It initializes all three machine learning models (KNN, LDA, SVM) using the Scikit-learn library.
 - It iterates through each model, training it on the preprocessed training data using the .fit() method.
 - Each trained model is then evaluated on the preprocessed, unseen test data using the .predict() method.
 - The performance metrics (Accuracy, Precision, Recall, F1-Score) for each model are calculated and stored.
 - These performance results are saved to a file named model_performance.csv.
 - The script identifies the best-performing model based on the highest accuracy.
 - Finally, it saves the best model (nids_model.pkl), the data scaler (scaler.pkl), and the list of training columns (model_columns.pkl) to disk using joblib for later use by the web application.

B. Interactive Dashboard (app.py)

- Technology:
 - The front-end application is built entirely in Python using the Streamlit framework. This was chosen for its capacity for rapid development and the ability to create intuitive, data-centric user interfaces without writing HTML/CSS/JavaScript.
- Core Functionality:
 - Asset Loading: Upon startup (streamlit run app.py), the application loads the saved assets: nids_model.pkl, scaler.pkl, model_columns.pkl, and model_performance.csv.
 - User Interface: The UI presents a clean, two-column dashboard layout. The main analysis area is kept clean until a file is uploaded.
 - File Uploader: The primary interactive element is st.file_uploader(), which allows the user to select and upload one or more CSV files containing network traffic data.
 - Conditional Display: The main analysis content, including the model performance table and prediction results, is conditionally rendered and only appears *after* a file has been successfully uploaded.
 - Real-Time Prediction: Once a file is uploaded, the application:
 - Reads the CSV into a Pandas DataFrame.

- Applies the same preprocessing steps (encoding and scaling) using the loaded assets to ensure consistency.
- Feeds the processed data into the loaded `nids_model.pkl` to generate predictions.
- Results Visualization: The results are displayed in a user-friendly, tabbed format:
 - Prediction Summary Tab: Shows high-level metrics (total connections, attacks found) and a clear `st.warning()` or `st.success()` message.
 - Detailed Results Tab: Presents a scrollable table with the original data and an added 'Prediction' column. Rows predicted as 'Attack' are highlighted in red for easy identification.
 - Input Data Tab: Shows a preview of the raw data the user uploaded.
- Unique Feature: The dashboard also displays the static `model_performance.csv` table, providing context to the user about *why* a particular model was chosen for the predictions, which adds a layer of transparency.



V. RESULTS AND DISCUSSION

A. Performance Metrics

- To quantitatively and objectively evaluate the models, the following standard classification metrics were used¹³³:
- Accuracy:
 - The ratio of correctly predicted instances to the total number of instances¹³⁴.
 - It provides a general measure of the model's overall performance¹³⁵.
 - *Formula*: $\$Accuracy = (TP + TN) / (TP + TN + FP + FN)$ ¹³⁶
- Precision:
 - The ratio of correctly predicted positive instances (attacks) to the total number of instances predicted as positive¹³⁷.
 - High precision indicates a low false positive rate, which is crucial for avoiding alert fatigue¹³⁸.
 - *Formula*: $\$Precision = TP / (TP + FP)$ ¹³⁹
- Recall (Sensitivity):
 - The ratio of correctly predicted positive instances to the total number of *actual* positive instances¹⁴⁰.
 - High recall indicates a low false negative rate, meaning the model is effective at finding most of the real attacks¹⁴¹.
 - *Formula*: $\$Recall = TP / (TP + FN)$ ¹⁴²
- F1-Score:
 - The harmonic mean of Precision and Recall¹⁴³.

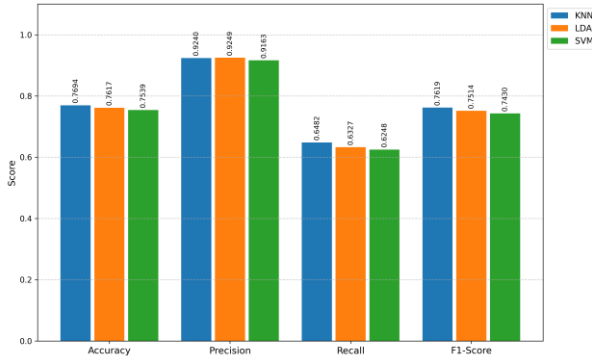
- It provides a single, balanced score, which is particularly useful for imbalanced datasets where one metric might be misleading¹⁴⁴.
- *Formula:* $F1\text{-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$.

B. Comparative Analysis

- The three models were trained on the pre-processed KDDTrain+.txt dataset and evaluated on the unseen KDDTest+.txt dataset, yielding the following performance¹⁴⁶.

Model	Accuracy	Precision	Recall	F1-Score
KNN	0.7694	0.9240	0.6482	0.7619
LDA	0.7617	0.9249	0.6327	0.7514
SVM	0.7539	0.9163	0.6248	0.7430

Fig. 4. Visual Comparison of Model Performance Metrics on NSL-KDD Test Set



C. Discussion of Results

- Overall Performance:
 - All three models demonstrated a high level of performance, with accuracies exceeding 75% and precision scores over 91%¹⁴⁸.
 - This confirms the viability of using classical machine learning algorithms as a powerful and effective foundation for an anomaly-based NIDS¹⁴⁹.
- Best Performing Model:
 - The K-Nearest Neighbours (KNN) model achieved the highest Accuracy (76.94%) and the highest F1-Score (76.19%)¹⁵⁰.
 - This suggests that in the high-dimensional feature space of the NSL-KDD dataset, the instance-based classification approach of KNN was the most effective overall at balancing the competing needs of precision and recall¹⁵¹.
- Precision vs. Recall:
 - The Linear Discriminant Analysis (LDA) model achieved the highest Precision (92.49%)¹⁵².
 - This indicates that LDA was the best at avoiding false alarms (false positives). In a real-world scenario, this is highly valuable as it reduces alert fatigue for security analysts¹⁵³.
 - However, this high precision came at the cost of a slightly lower Recall, meaning it missed more

actual attacks (false negatives) compared to KNN¹⁵⁴.

- SVM Performance:
 - The Support Vector Machine, using a linear kernel, showed slightly lower performance across all metrics compared to KNN and LDA in this specific experiment¹⁵⁵.
 - This could be attributed to the choice of a linear kernel, suggesting that the decision boundary between 'Normal' and 'Attack' in this dataset may not be purely linear¹⁵⁶. Performance could potentially be improved with a non-linear kernel (e.g., RBF)¹⁵⁷.
- Conclusion on Model Selection:
 - Based on the balanced performance indicated by the highest F1-Score and the highest overall Accuracy, the KNN model was selected as the best-performing model for deployment in the Streamlet dashboard (nids_model.pkl)¹⁵⁸.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

- Summary of Achievements:
 - This project successfully demonstrated the entire lifecycle of a machine learning project, from data exploration and preprocessing to model training, evaluation, and deployment as a practical web application.
- Empirical Validation:
 - It was empirically shown that classical machine learning algorithms like KNN, LDA, and SVM can serve as a powerful foundation for building an effective anomaly-based Network Intrusion Detection System.
- Model Selection:
 - Through a rigorous comparative analysis, the K-Nearest Neighbours (KNN) algorithm was identified as the most effective model for the binary classification task on the NSL-KDD dataset, achieving the best balance of performance metrics (Highest Accuracy and F1-Score).
- Practical Contribution:
 - The development of the Streamlet dashboard provides a tangible proof-of-concept, bridging the gap between theoretical machine learning models and practical, interactive tools for security analysis.

B. Future Work

- Integration of Deep Learning Models:
 - The initial project scope included an analysis of deep learning models. Future work should focus on implementing and comparing models like Multi-Layer Perceptron's (MLP), Long Short-Term Memory (LSTM) networks, and Autoencoders to investigate if they can provide superior performance, especially in detecting complex, time-series-based attacks.
- Testing on Modern Datasets:

- To ensure relevance against contemporary cyber threats, the models should be re-evaluated on more modern and comprehensive datasets, such as CIC-IDS2017 or UNSW-NB15, which include traffic from modern network protocols and attack vectors.
- Development of a Real-Time Pipeline:
 - The current dashboard relies on batch processing of uploaded CSV files. A significant future enhancement would be to develop a real-time detection pipeline that can capture live network traffic using libraries like Scapy or Pyshark, preprocess it on the fly, and make immediate predictions.
- Multi-Class Classification:
 - The current system performs binary classification ('Normal' vs. 'Attack'). Future work could expand the model to perform multi-class classification to not only detect an intrusion but also identify the specific *type* of attack (e.g., DoS, Probe, R2L, U2R), providing more actionable intelligence to security analysts.
- Hyperparameter Optimization:
 - A systematic hyperparameter tuning process, using techniques like Grid Search or Randomized Search, could be performed for each model to potentially extract even higher performance.

VII. REFERENCES

- [1] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 dataset," in Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, Ottawa, ON, Canada, 2009, pp. 1-8.
- [2] S. Revathi and A. Malathi, "A Detailed Analysis on NSL-KDD Dataset Using Various Machine Learning Techniques for Intrusion Detection," International Journal of Engineering Research & Technology (IJERT), vol. 2, no. 12, pp. 279-284, 2013.
- [3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), Funchal, Madeira, Portugal, 2018, pp. 108-116.
- [4] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in 2015 Military Communications and Information Systems Conference (MilCIS), Canberra, ACT, Australia, 2015, pp. 1-6.
- [5] J. H. Park and J. H. Park, "Wireless Intrusion Detection System Using Machine Learning," in Human-centric Computing and Information Sciences, vol. 7, no. 1, 2017, Art. no. 43.
- [6] A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security," IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153-1176, Secondquarter 2016.
- [7] S. M. H. Adnan, M. N. A. Khan, and F. A. Barbhuiya, "A survey on intrusion detection using machine learning," in 2017 4th International Conference on Advances in Electrical Engineering (ICAEE), Dhaka, Bangladesh, 2017, pp. 448-453.
- [8] C. Yin, Y. Zhu, J. Fei, and X. He, "A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks," IEEE Access, vol. 5, pp. 21954-21961, 2017.
- [9] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep Learning Approach for Intelligent Intrusion Detection System," IEEE Access, vol. 7, pp. 41565-41587, 2019.
- [10] C. Ieracitano, A. Adeel, F. C. Morabito, and A. Hussain, "A novel statistical analysis and autoencoder driven intelligent intrusion detection approach," Neurocomputing, vol. 387, pp. 51-62, 2020.
- [11] M. A. Ambusaidi, X. He, P. Nanda, and Z. Tan, "Building an effective intrusion detection system using a filter-based feature selection algorithm," IEEE Transactions on Computers, vol. 65, no. 10, pp. 2986-2998, Oct. 2016.
- [12] D. E. Denning, "An Intrusion-Detection Model," IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222-232, Feb. 1987.
- [13] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [14] W. McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, Austin, TX, USA, 2010, pp. 51-56.
- [15] C. B. C. Team, "Streamlit: The fastest way to build custom ML tools," Streamlit Inc., 2019. [Online]. Available: <https://streamlit.io>
- [16] A. Amri, "joblib: running Python functions as pipeline jobs," 2010. [Online]. Available: <https://joblib.readthedocs.io/>
- [17] M. Sabhnani and G. Serpen, "Why Machine Learning Algorithms Fail in Misuse Detection on KDD Intrusion Detection Data Set," Intelligent Data Analysis, vol. 8, no. 4, pp. 403-415, 2004.
- [18] S. T. Powers and J. He, "A survey of intrusion detection systems," Potentials, IEEE, vol. 22, no. 3, pp. 18-21, 2003.
- [19] D. S. Kim and J. S. Park, "Network-based intrusion detection with support vector machines," in Proceedings of the

- International Conference on Information Networking, vol. 2, 2003, pp. 747-756.
- [20] A. Sung and S. Mukkamala, "Identifying important features for intrusion detection using support vector machines and neural networks," in *Proceedings of the 2003 Symposium on Applications and the Internet*, Orlando, FL, USA, 2003, pp. 209-216.
- [21] K. M. Faraoun and A. Boukelif, "Neural Networks Learning for Computer Security," *Computer Systems Science and Engineering*, vol. 21, no. 5, pp. 303-311, 2006.
- [22] H. G. Kayacik, A. N. Zincir-Heywood, and M. I. Heywood, "On the capability of an SOM based intrusion detection system," in *The 2003 International Joint Conference on Neural Networks*, vol. 3, 2003, pp. 1808-1813.
- [23] Y. Li, "An AdaBoost-based algorithm for network intrusion detection," in *2010 3rd International Conference on Computer Science and Information Technology*, Chengdu, China, 2010, vol. 9, pp. 119-123.
- [24] M. Panda and M. R. Patra, "Network Intrusion Detection Using Naive Bayes," *International Journal of Computer Science and Network Security*, vol. 7, no. 12, pp. 258-263, 2007.
- [25] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vazquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Computers & Security*, vol. 28, no. 1-2, pp. 18-28, 2009.
- [26] A. A. Ghorbani, W. Lu, and M. Tavallaee, *Network Intrusion Detection and Prevention: Concepts and Techniques*. New York, NY, USA: Springer, 2010.
- [27] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.
- [28] H. Liao, C. R. Lin, Y. Lin, and K. Tung, "Intrusion detection system: A comprehensive review," *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16-24, 2013.
- [29] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.
- [30] C. C. Aggarwal, *Data Mining: The Textbook*. Cham, Switzerland: Springer, 2015.
- [31] A. K. Jain, M. N. Murty, and P. J. Flynn, "Data Clustering: A Review," *ACM Computing Surveys*, vol. 31, no. 3, pp. 264-323, Sep. 1999.
- [32] Y. Bengio, A. Courville, and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798-1828, Aug. 2013.
- [33] G. E. Hinton and R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*, vol. 313, no. 5786, pp. 504-507, 2006.
- [34] T. A. L. S. D. Santos, J. D. S. da Silva, A. H. R. Costa, and G. H. G. D. da Costa, "A survey of network-based intrusion detection systems," *Journal of Network and Computer Applications*, vol. 106, pp. 20-36, 2018.
- [35] P. Lichman, "UCI Machine Learning Repository," Irvine, CA: University of California, School of Information and Computer Science, 2013. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [36] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [37] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436-444, 2015.
- [38] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770-778.
- [40] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems 27*, 2014, pp. 2672-2680.
- [41] J. Brownlee, *Machine Learning Mastery with Python*. San Juan, PR, USA: Machine Learning Mastery, 2016.
- [42] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [43] S. Raschka and V. Mirjalili, *Python Machine Learning*, 3rd ed. Birmingham, UK: Packt Publishing, 2019.
- [44] F. Chollet et al., "Keras," 2015. [Online]. Available: <https://keras.io>
- [45] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015, software available from tensorflow.org.
- [46] O. Y. Al-Jarrah, Y. Al-Hammouri, E. A. A. Al-Dujaili, and M. A. A. Al-Shawabka, "An efficient intrusion detection system for computer networks using Gini index and random

forest," *Journal of Computer Science*, vol. 11, no. 4, pp. 586-594, 2015.

[47] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[48] R. A. Fisher, "The Use of Multiple Measurements in Taxonomic Problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179-188, 1936.

[49] A. P. St-Gelais, J. S. S. M. Wong, and J. M. Fernandez, "A Systematic Literature Review of Machine Learning Methods for Intrusion Detection in Industrial Control Systems," *IEEE Access*, vol. 8, pp. 173038-173059, 2020.

[50] E. Hodo, X. Bellekens, A. Hamilton, C. Tachtatzis, and R. Atkinson, "Threat analysis of IoT networks using artificial neural network," in *2016 International Symposium on Networks, Computers and Communications (ISNCC)*, Hammamet, Tunisia, 2016, pp. 1-6.